

VLAConf: Calibrated Task-Success Confidence for Vision-Language-Action Models

Dehao Huang^{1,2}, Aoxiang Gu¹, Chengjie Zhang¹, Bolin Zou¹,
Wenlong Dong¹, Zilang Cen^{2,3}, Yue Wang², and Hong Zhang¹

Abstract— Confidence estimation for Vision-Language-Action (VLA) models is essential for robots to perform manipulation tasks in the open world, providing crucial signals for risk-sensitive decision-making and failure anticipation. Existing confidence estimation methods typically rely on ensemble-based paradigms or action-token probabilities to predict the likelihood of task success. However, they still encounter challenges in computational efficiency and cross-architecture generalizability. These methods usually require repeated sampling, leading to inference inefficiency, and are restricted to VLA models with discrete action outputs, making them difficult to apply to continuous action spaces. To address this issue, we propose VLAConf, a one-class discriminative confidence framework. By leveraging frozen pretrained VLA internal representations, VLAConf directly estimates step-wise anomaly scores in a single forward pass using a lightweight confidence head, thereby eliminating the overhead of exhaustive resampling. We additionally use step-conditioned modeling to encode rollout-phase information along the manipulation trajectory. Experiments on the LIBERO benchmark demonstrate that VLAConf significantly improves the quality of the confidence signal constructed for post-hoc calibration, outperforming existing baselines by a large margin in inference efficiency. The effectiveness of VLAConf is further validated in real-robot experiments. To access the source code and supplementary videos, visit <https://sites.google.com/view/vlaconf>.

I. INTRODUCTION

Confidence estimation is essential for deploying Vision-Language-Action (VLA) models in open-world robotic manipulation, as illustrated in Figure 1. Although VLA models have shown strong cross-task generalization, real-world execution still involves distribution shifts, long-tail scenarios, and unexpected failures. Therefore, a robot should not only generate control actions, but also predict how likely it is to successfully complete the current task. Such confidence supports risk-sensitive decision-making, failure anticipation, action rollback, and human-robot collaboration. Motivated by the need for a reliable estimate of task success itself, we study how to derive a success-relevant signal from pretrained VLA representations and calibrate it into task-success confidence for reliable evaluation.

Existing VLA confidence methods predominantly follow an ensemble-based paradigm, where confidence is derived from predictive disagreement across multiple models or prompts and then calibrated post hoc. Although intuitive,

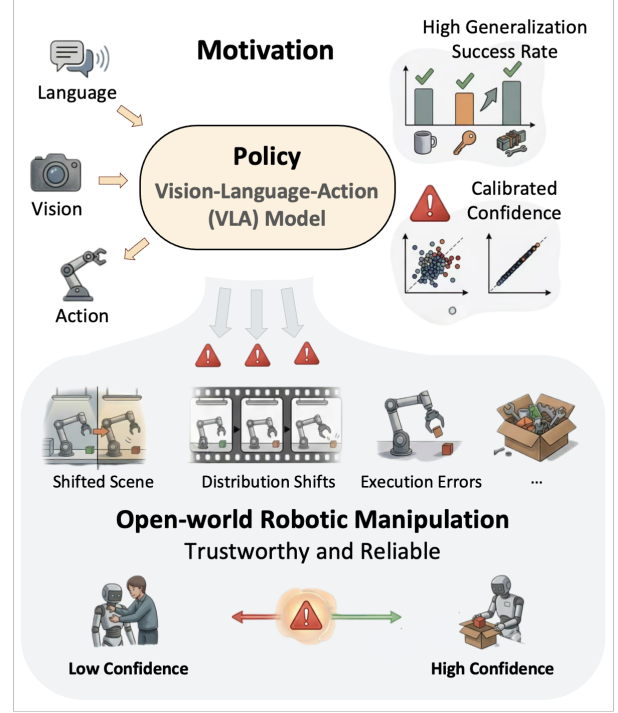


Fig. 1. Motivation of VLAConf. Despite strong generalization, VLA policies can still encounter shifted scenes, distribution shifts, and execution errors in open-world manipulation, highlighting the need for calibrated task-success confidence to distinguish low-confidence situations from high-confidence executions.

this paradigm is difficult to deploy in modern VLA systems: repeated sampling introduces substantial computational overhead under real-time control constraints, and disagreement over continuous action outputs can be unstable and difficult to interpret as task-level success likelihood. A complementary line of uncertainty estimation and failure detection research in robot manipulation has explored one-class and distance-based formulations, which can avoid repeated sampling and provide direct scores for continuous-control contexts. However, these methods have largely been studied in task-specific visuomotor policies, leaving it unclear how to extend them to generalist VLAs with heterogeneous visual and linguistic inputs.

To address this challenge, we propose VLAConf, a one-class discriminative confidence framework that leverages frozen pretrained VLA representations to construct an efficient confidence signal for robotic manipulation. Instead of using repeated sampling and post hoc disagreement aggregation, VLAConf estimates confidence in a single forward pass using a lightweight confidence head built on top of

¹Department of Electronic and Electrical Engineering, Southern University of Science and Technology, Shenzhen, China.

²Zhongguancun Academy, Beijing, China

³National Cybersecurity Academy, Wuhan University, Wuhan, China.

This work is supported by the Zhongguancun Academy (Grant No. C20250203).

the pretrained VLA backbone. Concretely, it pools visual and language hidden states from the VLA, incorporates proprioceptive state to encode the current execution context, and trains a Coin-Flip Network objective to learn a one-class scoring function over the resulting representation space, such that typical training-distribution contexts are associated with low-norm coin vectors while atypical contexts yield larger scores. To account for the temporal structure of manipulation, the scoring head is step-conditioned so that the same execution context can be interpreted relative to rollout phase. The resulting confidence signal is efficient, transferable, and backbone-agnostic, and can be calibrated into task-success confidence for reliable VLA deployment.

II. RELATED WORK

A. Confidence Estimation and Uncertainty in VLAs

Recent vision-language-action (VLA) models have demonstrated strong semantic grounding and broad manipulation capabilities by combining pretrained vision-language backbones with action generation modules [1]–[5]. Despite this progress, evaluating the reliability of VLA systems during deployment remains challenging. In the broader context of large language and vision-language models, uncertainty quantification often relies on token-level probabilities, semantic entropy, or generation consistency [6]–[10].

Inspired by these approaches, initial efforts toward confidence estimation for VLAs have predominantly followed an ensemble-based paradigm [11]–[13]. Notably, ConfidenceVLA [14] shares our goal of providing calibrated task-success confidence for VLA models. They achieve this by deriving confidence through predictive disagreement across prompt ensembles and action-token probabilities. While conceptually appealing, this strategy faces two critical limitations in modern robotic deployment. First, repeated sampling and ensemble inference are computationally expensive, making them difficult to reconcile with the high-frequency, real-time control requirements of physical robots. Second, relying on action-token probabilities restricts these methods to discrete-action VLAs. For state-of-the-art VLAs that output continuous actions (e.g., via diffusion or flow-matching [1]), disagreement-based uncertainty is often unstable and difficult to interpret directly as a success likelihood. To achieve an efficient, continuous-action-compatible confidence signal, we must shift our focus from the action-output space to the model’s internal representation space.

B. Out-of-Distribution and Failure Detection in Robotics

Extracting reliability signals from internal representations is deeply connected to out-of-distribution (OOD) and failure detection in robot manipulation. A practical regime assumes that failures are scarce, expensive, or unsafe to collect, meaning successful executions provide the only reliable supervision. Traditional OOD methods—such as density estimation, one-class discriminators, distance metrics, and consistency checks [15]–[21]—excel in this regime because

they are strictly *success-only*. Recent works, such as FAIL-Detect [16], systematically evaluate these methods for generative imitation learning policies. We share this success-only philosophy, but these methods have largely been designed for task-specific visuomotor policies. Scaling them to generalist VLAs introduces a significant multi-task generalization challenge, as the diverse visual scenes and novel language instructions inherent to generalist deployment can easily be misclassified as OOD anomalies.

To handle the complexity of generalist policies, a separate line of work explores supervised failure detection [22]–[25]. For instance, SAFE [26] demonstrates that the internal latent features of VLAs capture rich, task-agnostic semantics about execution success, and successfully trains a multi-task failure classifier on top of these representations. While this resolves the multi-task generalization issue, it fundamentally relies on collecting diverse *failed* rollouts for supervised training, which violates the practical success-only constraint and poses significant safety risks in real-world open-vocabulary manipulation.

Positioning of Our Approach. Our work, VLAConf, synthesizes these perspectives to provide a practical confidence signal for modern VLAs. Like ConfidenceVLA [14], our primary objective is to produce calibrated task-success confidence. However, we achieve this in a single forward pass without relying on discrete action tokens or expensive ensembles. To do so, we adopt the *success-only* philosophy of traditional OOD methods [16] by formulating confidence estimation as a one-class discriminative problem. However, we apply it directly to the frozen, rich latent representations of VLAs—similar to supervised methods like SAFE [26]—to ensure multi-task generalization. By training a lightweight confidence head with a Coin-Flip Network objective on successful rollouts alone, VLAConf bridges these paradigms, delivering an efficient, backbone-agnostic confidence measure naturally suited for continuous-action generalist policies.

III. PROBLEM DEFINITION

We study task-success confidence estimation for deployed vision-language-action (VLA) policies in language-conditioned robotic manipulation, as shown in the upper part of Figure 2. Given a language instruction l and an observed execution prefix $\rho_t = (o_1, \dots, o_t)$, where $o_t = (I_t, x_t)$ contains the current image and proprioceptive state, the goal is to estimate the probability that the current rollout will eventually succeed under the deployed policy, $p(Y = 1 \mid \rho_t, l)$, where $Y \in \{0, 1\}$ denotes the final rollout-level task outcome. This formulation covers both PRE-EXECUTION confidence, where only the initial observation is available, and ONLINE EXECUTION confidence, where the estimate is updated as the rollout unfolds. The latter is important for manipulation because task progress and emerging failures are often only visible after execution begins.

We assume a practical imitation-learning workflow. The VLA policy is first pretrained on broad robot data and then adapted to the target task using successful demonstrations, denoted by $\mathcal{D}_{\text{sft}}$. For confidence learning, we use the

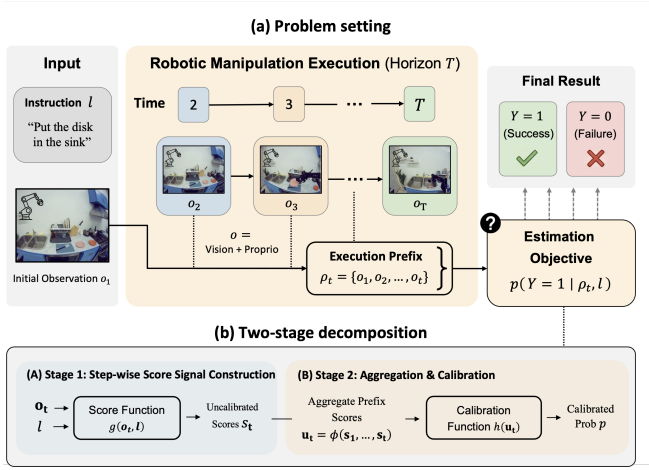


Fig. 2. Problem formulation for VLA confidence estimation. Top: given an instruction and an observed rollout prefix, the goal is to estimate the probability that the current execution will eventually succeed. Bottom: VLAConf decomposes this problem into step-wise score construction followed by prefix aggregation and calibration.

observation-language traces of these demonstrations, written as $\tau_i = (o_{i,1:T_i}, l_i)$, where l_i is the language instruction and $o_{i,k} = (I_{i,k}, x_{i,k})$. Action labels used for policy adaptation are omitted from the notation because VLAConf keeps the policy parameters fixed and learns confidence on top of frozen VLA representations. These demonstrations are not collected for confidence learning and contain no failed executions.

In addition, a small set of completed policy rollouts can be labeled with binary task outcomes; we denote this outcome-labeled set by $\mathcal{D}_{\text{cal}} = \{(\rho^{(j)}, l^{(j)}, Y^{(j)})\}_{j=1}^{N_{\text{cal}}}$. VLAConf uses the two sources separately: $\mathcal{D}_{\text{sft}}$ defines the support of normal successful execution and is used to learn an uncalibrated anomaly signal, while $\mathcal{D}_{\text{cal}}$ is used only to calibrate that scalar signal into a task-success probability. Thus, representation-level confidence learning remains success-only, and scarce outcome labels are reserved for low-capacity probability calibration.

As summarized in the lower part of Figure 2, VLAConf first produces step-wise anomaly scores and aggregates the observed prefix into a scalar signal u_t . It then fits a calibration map $h(\cdot)$, implemented as Platt scaling in our experiments, to convert this signal into $p(Y = 1 | \rho_t, l) = h(u_t)$.

IV. VLAConf

A. Overview

An overview of VLAConf is shown in Figure 3. Given a rollout prefix $\rho_t = (o_1, \dots, o_t)$ and instruction l , VLAConf estimates task-success confidence by measuring whether the observed execution remains close to the success manifold represented by $\mathcal{D}_{\text{sft}}$. The method first extracts a step-wise execution representation from a frozen VLA, maps each step to an *anomaly score*—following standard one-class OOD detection terminology, a non-negative scalar that grows when the input falls outside the support of successful executions—with a step-conditioned scoring head, and finally aggregates and calibrates these scores into $p(Y = 1 | \rho_t, l)$.

We first describe how VLAConf constructs backbone-agnostic execution features from frozen VLA hidden states. We then introduce the step-conditioned anomaly scoring objective trained only on successful trajectories. Finally, we present prefix-level aggregation and post-hoc calibration, which convert step-wise anomaly scores into calibrated task-success probabilities.

B. Frozen VLA Execution Representation

The first step is to obtain a representation on which support can be estimated consistently across VLA architectures. We estimate confidence in the *internal representation space* of a pretrained VLA rather than in its action-output space. This is important because manipulation failures are frequently reflected by whether the current multimodal context departs from familiar successful behavior, not merely by whether the immediate action prediction appears sharp. Moreover, action probabilities are often unavailable or hard to compare across architectures, especially when the action head is continuous rather than token based.

Let f_θ denote a pretrained VLA backbone. We keep f_θ frozen throughout confidence learning. This choice preserves the semantic and geometric priors already learned by the VLA and keeps confidence estimation efficient because only a small head is optimized. Thus, VLAConf treats the pretrained VLA as a general-purpose multimodal feature extractor and learns confidence on top of that representation rather than by adapting the policy itself.

At step k , the backbone processes the current observation-instruction pair (o_k, l) and returns hidden states $\mathcal{H}_k$. Although different backbones expose these hidden states through different tensor layouts, VLAConf only assumes a shared pooled-feature interface. We separately pool the valid visual tokens and language tokens, yielding a visual summary h_k^v and a language summary h_k^l . These summaries capture the scene content and task semantics needed for confidence estimation. We additionally fuse the robot proprioceptive state because visual-language context alone may not fully determine whether the current execution phase is healthy. Variables such as gripper opening and end-effector pose often distinguish between a normal state and an abnormal one even when the image appears similar. We project the proprioceptive component x_k with a small state encoder and fuse it with the pooled visual-language features:

$$\begin{aligned} \mathcal{H}_k &= f_\theta(o_k, l), \\ h_k^v &= \text{Pool}_v(\mathcal{H}_k), \quad h_k^l = \text{Pool}_l(\mathcal{H}_k), \\ h_k^x &= E_x(x_k), \quad z_k = M_{\text{mix}}([h_k^v; h_k^l; h_k^x]), \end{aligned}$$

where $\text{Pool}_v(\cdot)$ and $\text{Pool}_l(\cdot)$ denote masked mean pooling over the valid visual and language token positions, $E_x(\cdot)$ is a small MLP state encoder, and $M_{\text{mix}}(\cdot)$ is a two-layer mixing MLP. The result is a compact execution representation z_k on which confidence is learned. This representation hides backbone-specific implementation details while retaining the visual, semantic, and proprioceptive information needed to assess familiarity.

C. Step-Conditioned Anomaly Scoring

Given the execution feature z_k , we next learn a scalar anomaly score without requiring failure labels. We build this score on the Coin-Flip Network (CFN) of Lobel et al. [27], which fits a small head $q(\cdot)$ to predict an i.i.d. Rademacher target $c \sim \{-1, +1\}^d$ for every input. Under the squared-error optimum, the network output for an input z encountered $N(z)$ times collapses to the empirical mean of its targets, meaning its expected squared norm is proportional to $1/N(z)$. The squared norm is therefore a smooth anomaly score: features common in the training distribution yield small norms, while atypical features yield large ones. This makes CFN a natural fit for our success-only setting.

Importantly, manipulation confidence is inherently phase-dependent: the same scene may correspond to normal behavior at one stage of a task and abnormal behavior at another (e.g., an open gripper is expected before grasp but suspicious after lift). Therefore, we condition the anomaly scoring head on the current rollout step, allowing the model to learn phase-specific notions of familiarity.

To encode task phase, we introduce a step encoder that maps the rollout step index k to a compact descriptor. Let K be a fixed normalization horizon and let $\hat{k} = \text{clip}(k-1, 0, K-1)$. The descriptor combines a learned step embedding with a normalized progress scalar:

$$\psi_k = [e_k; \bar{k}], \quad e_k = \text{Emb}(\hat{k}), \quad \bar{k} = \hat{k}/(K-1),$$

where $\text{Emb}(\cdot)$ is a learned embedding table of size K . Using a fixed K rather than the per-trajectory length T_i keeps $\bar{k}$ well-defined in the online setting, where the eventual rollout length is unknown; the clip ensures both e_k and $\bar{k}$ stay in range when k exceeds $K-1$ at inference. The resulting descriptor captures both discrete phase identity and coarse relative progress through the rollout.

Instead of concatenating ψ_k with z_k , we use the descriptor to *modulate* the anomaly scoring head with a FiLM-style affine transform [28] followed by a residual gate, allowing the same execution representation to be interpreted differently at different phases of the task. Let $W_{\text{proj}}(\cdot)$ denote a Linear $\rightarrow$ LN $\rightarrow$ Tanh pre-projection, $A_{\text{cond}}(\cdot)$ a two-layer MLP that encodes ψ_k , and $q(\cdot)$ a two-layer MLP coin-value head. We write

$$\begin{aligned} h_k &= W_{\text{proj}}(z_k), & c_k &= A_{\text{cond}}(\psi_k), \\ (\gamma_k, \beta_k) &= W_{\text{film}}(c_k), & \eta_k &= \sigma(W_{\text{gate}}(c_k)), \\ \tilde{h}_k &= \text{LN}((1 + \gamma_k) \odot h_k + \beta_k), \\ \tilde{z}_k &= \eta_k \odot \tilde{h}_k + (1 - \eta_k) \odot h_k, \\ v_k &= q(\tilde{z}_k), & s_k &= \|v_k\|_2^2, \end{aligned}$$

where $\sigma(\cdot)$ is the sigmoid, $\odot$ denotes element-wise multiplication, W_{film} and W_{gate} are independent linear maps that share the upstream conditioning network A_{cond} , and the post-affine LN stabilizes the modulated branch before gating. The modulated feature $\tilde{z}_k$ is then mapped to a vector $v_k \in \mathbb{R}^d$, whose squared norm defines the step-wise anomaly score s_k .

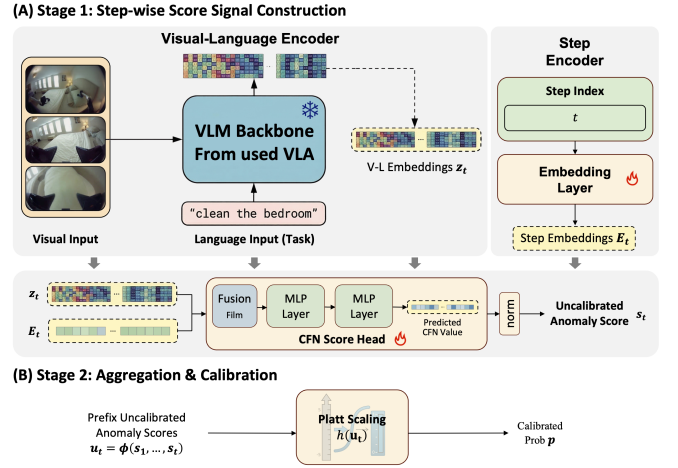


Fig. 3. Overview of VLAConf. For each observed step, Stage 1 extracts frozen VLA features and applies a step-conditioned head to construct a step-wise anomaly score $s_k = g(o_k, l, k)$. Over a rollout prefix, Stage 2 aggregates the available scores into an uncalibrated prefix-level signal $u_t = \phi(s_1, \dots, s_t)$ and converts it into task-success confidence by post-hoc calibration, $p(Y = 1 | \rho_t, l) = h(u_t)$. The full model’s proprioceptive vision branch is omitted for clarity.

We train the head with a success-only objective over the step-wise samples extracted from successful rollouts:

$$\mathcal{D}_{\text{step}}^+ = \{(o_{i,k}, l_i, k) : \tau_i \in \mathcal{D}_{\text{sft}}, k = 1, \dots, T_i\}.$$

For each sample, the head predicts a vector $v_k \in \mathbb{R}^d$. Instead of using success labels, each sample is assigned a deterministic pseudo-random target $c_{i,k} = \Gamma(i, k) \in \{-1, +1\}^d$, where Γ is a deterministic generator. The head is then optimized by mean squared error:

$$\mathcal{L}_{\text{CFN}} = \frac{1}{|\mathcal{D}_{\text{step}}^+|} \sum_{\tau_i \in \mathcal{D}_{\text{sft}}} \sum_{k=1}^{T_i} \|v_{i,k} - c_{i,k}\|_2^2.$$

At inference time, the squared norm $\|v_k\|_2^2$ is directly used as the step-wise anomaly score s_k , which preserves a useful ordering of in- vs. out-of-distribution evidence.

D. Prefix Aggregation and Calibration

The step-wise anomaly score s_k is local and uncalibrated, whereas the target quantity is rollout-level success probability. A robot may briefly visit an unusual state and still recover, or it may appear normal early in the rollout and fail only after a later deviation. Confidence should therefore summarize the evidence accumulated across the observed prefix rather than depend on a single step in isolation.

We therefore define a trajectory-level aggregated signal $u_t = \phi(s_1, \dots, s_t)$, where $\phi(\cdot)$ aggregates the step-wise scores up to time t . The most suitable aggregation rule depends on the deployment protocol. In the pre-execution setting, only the initial observation is available, so we set $u_1 = s_1$. In the online setting, we consider simple aggregation rules such as the running mean, $u_t = \frac{1}{t} \sum_{k=1}^t s_k$, or the maximum prefix score. This modularity allows the same step-wise model to support both one-shot risk assessment before execution and continuous confidence tracking during rollout.

The aggregated signal u_t is an uncalibrated scalar. To convert it into an interpretable success probability, we perform post-hoc calibration on a small set of outcome-labeled policy rollouts, $\mathcal{D}_{\text{cal}}$ (as introduced in Section III). This is the only stage of VLAConf that consumes outcome labels, requiring only a binary outcome $Y \in \{0, 1\}$ per rollout rather than dense per-step annotations.

We fit a standard Platt scaling model [29] to map the aggregated signal to a success probability:

$$h(u_t) = \text{sigmoid}(-\alpha u_t + \beta),$$

where $\alpha \geq 0$ and $\beta \in \mathbb{R}$ are learned by minimizing the binary cross-entropy on $\mathcal{D}_{\text{cal}}$. The explicit minus sign encodes the prior that larger anomaly scores indicate lower probability of success.

Overall, the pipeline adds only one MLP head and one scalar logistic regression per step, with negligible overhead relative to the VLA backbone and no additional sampling.

V. EXPERIMENTS

A. Experimental Setup

Backbones. We evaluate VLAConf on two representative VLA backbones spanning the discrete- and continuous-action regimes: (i) OPENVLA-OFT [1], a token-based VLA that produces actions through a discrete autoregressive head; and (ii) $\pi^{0.5}$ [2], a state-of-the-art generalist VLA that emits continuous actions via flow-matching. Both backbones are kept frozen during confidence learning. This pair lets us examine whether the same confidence signal can be constructed across heterogeneous action heads, and is also the setting under which prior action-token-based confidence methods break down.

Benchmarks. We use the LIBERO benchmark suite [30] for in-distribution evaluation, covering three subsets that stress different aspects of manipulation: LIBERO-GOAL, LIBERO-SPATIAL, and LIBERO-OBJECT. For robustness under shifted conditions, we additionally evaluate on LIBERO-PRO [31] and LIBERO-PLUS [32], which extend LIBERO with perturbations and previously unseen tasks (Section V-C). Real-robot results are reported in Section V-D.

Baselines. We compare against three external reference points and one ablation. (i) PCA-KMEANS: a post-hoc clustering OOD baseline adapted from robot execution failure detection [33]. It projects frozen VLA features with PCA, clusters successful demonstration features with K-means, and uses the distance to the nearest centroid as an anomaly score. (ii) TOKENPROB: the policy’s own action-token probability, aggregated over the action sequence. This lower-cost reference re-uses information already produced by the VLA. (iii) CONFIDENCEVLA [14]: a prompt-ensemble confidence method that derives task-success probability from action-token agreement across resampled prompts. TokenProb and ConfidenceVLA rely on per-token action probabilities and are therefore restricted to VLAs with discrete action heads; they are not directly applicable to continuous-action backbones such as $\pi^{0.5}$, and the corresponding entries are marked

“/” in Table I. (iv) VLAConf-NOSTEP: an ablation of VLAConf that directly transfers prior one-class discriminator ideas, such as RND and CFM-style success-only scoring [34], [35], to frozen VLA representations. It uses the same frozen VLA features and one-class scoring objective as VLAConf, but removes the step encoder, isolating the effect of phase conditioning.

Evaluation protocol. For each deployed policy, we reserve a small validation set of completed rollouts as $\mathcal{D}_{\text{cal}}$, used to fit the two Platt-scaling parameters. We then evaluate on disjoint held-out rollouts and record, per rollout, (a) the binary outcome $Y \in \{0, 1\}$, (b) the rollout length, and (c) confidence queries from each method. Confidence is queried in two regimes: PRE-EXECUTION, where only the initial observation o_1 is available, and ONLINE EXECUTION, where the prefix ρ_t accumulates over the rollout and the available step-wise scores are aggregated (Section IV). For scalar online-execution table entries, we report the fixed checkpoint at 50% of policy execution (using max aggregation), so the comparison reflects an in-progress estimate available during deployment.

Metrics. We report three calibration metrics that are standard for binary outcomes and capture complementary aspects of probability quality: EXPECTED CALIBRATION ERROR (ECE) measures the average gap between predicted confidence and empirical accuracy across confidence bins; BRIER SCORE measures the mean squared error between predicted probability and the binary outcome; and NEGATIVE LOG-LIKELIHOOD (NLL) penalizes overconfident wrong predictions more aggressively than Brier. Lower is better for all three. We additionally report task SUCCESS RATE of the underlying policy and the AVERAGE INFERENCE TIME per confidence query, measured on the same hardware.

Implementation details. The CFN head is a two-layer MLP applied on top of frozen VLA features, with masked-mean pooling over visual and language tokens and a small MLP encoder for proprioceptive state. The step descriptor consists of a learned embedding $\text{Emb}(\hat{k})$ together with a normalized progress scalar $\hat{k}/(K-1)$, where $\hat{k} = \text{clip}(k-1, 0, K-1)$ and K is a fixed hyperparameter ($K=96$ for $\pi^{0.5}$, $K=256$ for OpenVLA-OFT). The descriptor modulates the coin-scoring head through a FiLM affine transform with a post-affine LayerNorm and a sigmoid residual gate. Coin-vector dimension is $d = 64$. We train the head with AdamW for the same number of optimization steps across backbones and suites; the only stage that consumes outcome labels is the per-suite Platt scaling. Unless otherwise stated, temporal analyses use the running mean of step-wise scores, while the main scalar online results use max aggregation as specified above.

Evaluation organization. We organize experiments into standard LIBERO, shifted LIBERO-Pro/Plus, and real-robot evaluations.

B. Main Results

We first evaluate on the standard LIBERO suites, which provide a controlled in-distribution setting for comparing

TABLE I. MAIN COMPARISON AVERAGED OVER STANDARD LIBERO SUITES.

Backbone	Method	Pre-Execution			Online Execution			Succ. (%)	Avg. Inf. Time (ms)
		ECE ↓	Brier ↓	NLL ↓	ECE ↓	Brier ↓	NLL ↓		
OpenVLA-OFT	PCA-kmeans	0.3031	0.3103	1.0976	0.3689	0.3510	1.1169	76.9	63.7
	TokenProb	0.2123	0.2220	0.6352	0.0922	0.1764	0.5404	77.7	65.1
	ConfidenceVLA	0.0363	0.1702	0.5295	0.0276	0.1647	0.5041	77.7	712.9
	VLAConf-NoStep	0.0410	0.1691	0.5148	0.0661	0.1656	0.5063	76.9	64.5
	VLAConf	0.0340	0.1614	0.4991	0.1188	0.1073	0.3335	78.2	64.9
$\pi^{0.5}$	PCA-kmeans	0.4168	0.3428	1.1049	0.4053	0.3394	1.0630	91.3	163.5
	ConfidenceVLA	/	/	/	/	/	/	/	/
	VLAConf-NoStep	0.0579	0.0898	0.3387	0.0448	0.0884	0.3261	90.5	164.4
	VLAConf	0.0370	0.0821	0.3141	0.0515	0.0668	0.2501	91.3	164.7

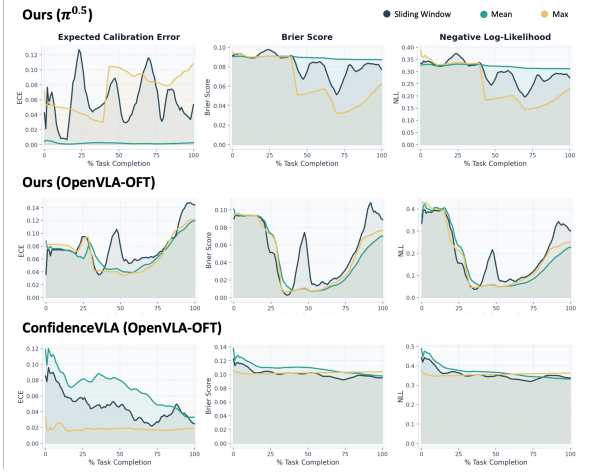


Fig. 4. Online calibration trends across task completion. ECE, Brier score, and NLL are evaluated across prefix lengths and aggregation rules.

confidence signals across backbones and execution regimes. Table I reports the average comparison over LIBERO-SPATIAL, LIBERO-OBJECT, and LIBERO-GOAL; the full suite-level results are provided in Appendix Table IV. Within each backbone group, bold marks the best applicable calibration metric. Small success-rate differences reflect stochasticity from independently repeated rollouts; the confidence estimators do not modify the underlying policy. Average inference time is measured per confidence query on the same hardware. We highlight three observations.

(1) **VLAConf improves probability quality while remaining efficient.** On OpenVLA-OFT, where all baselines are applicable, VLAConf offers the best trade-off between calibration quality and inference cost. TokenProb is the cheapest baseline because it directly reuses the policy’s own action-token probabilities, but its averaged Brier score and NLL are worse than those of VLAConf. PCA-kmeans is also consistently weaker, indicating that a simple clustering distance in frozen feature space is not sufficient. ConfidenceVLA is much more expensive because it repeatedly queries the VLA under a prompt ensemble, taking 712.9 ms per confidence query on average. In contrast, VLAConf adds only a lightweight head on top of frozen policy features and runs at 64.9 ms, essentially matching TokenProb (65.1 ms) while being about $11\times$ faster than ConfidenceVLA. Despite this efficiency gap, VLAConf is strongest under the proper scoring rules in the averaged OpenVLA-OFT comparison.

ConfidenceVLA still obtains the lowest online ECE, but this does not imply a more discriminative confidence signal: its prompt-ensemble score is concentrated and then Platt-scaled close to the empirical success rate, which can reduce bin-level calibration error while leaving little separation between successful and failed rollouts. The joint improvement on Brier score and NLL therefore provides stronger evidence that VLAConf produces sharper per-rollout probabilities rather than only matching the average success rate.

(2) **The same confidence head applies across action parameterizations.** VLAConf runs unchanged on both the discrete-action OpenVLA-OFT backbone and the continuous-action $\pi^{0.5}$ backbone, while TokenProb and ConfidenceVLA are unavailable for $\pi^{0.5}$ because they require discrete action-token probabilities. On $\pi^{0.5}$, VLAConf remains applicable on the LIBERO average, with pre-execution NLL of 0.3141 and online NLL of 0.2501. Compared with VLAConf-NoStep, it also gives lower Brier score and NLL in both regimes, showing that action-agnostic applicability alone is not enough; the representation-level score still benefits from phase conditioning. More broadly, because VLAConf only assumes access to frozen internal representations rather than a particular action parameterization, the same design can naturally extend to recent World Action Model policies that expose visual or dynamics features for control and planning [36], [37].

(3) **Online execution evidence is often helpful, but its benefit is not uniform.** Comparing the pre-execution and online columns in Table I, VLAConf benefits from observing the rollout prefix on OpenVLA-OFT: at the fixed-50% checkpoint, averaged Brier score and NLL both improve. The VLAConf-NoStep ablation further clarifies where this gain comes from. It keeps the same frozen features and success-only scoring objective, yet removing the step encoder weakens Brier score and NLL in most settings, especially in online OpenVLA-OFT results. This suggests that execution evidence must be interpreted relative to task progress rather than treated as an unordered set of familiar or unfamiliar states. Figure 4 shows a more nuanced temporal picture: some backbone-aggregator combinations improve substantially once task-relevant progress becomes visible, whereas others remain flat or degrade near the terminal stage. Thus, online evidence can sharpen confidence estimates, but the magnitude and timing of the benefit depend on both the backbone and the aggregation rule.

TABLE II. AVERAGE ROBUSTNESS.

Benchmark	Backbone	Method	ECE ↓	Brier ↓	NLL ↓	Success Rate
LIBERO-Pro	OpenVLA-OFT	ConfidenceVLA	0.0255	0.1645	0.4881	0.5055
	OpenVLA-OFT	VLAConf	0.0303	0.0452	0.1512	0.5037
	$\pi^{0.5}$	VLAConf	0.0200	0.0250	0.3100	0.6722
LIBERO-Plus	OpenVLA-OFT	ConfidenceVLA	0.0355	0.1752	0.5197	0.3396
	OpenVLA-OFT	VLAConf	0.0313	0.0325	0.1524	0.3405
	$\pi^{0.5}$	VLAConf	0.0433	0.0412	0.2824	0.5352



Fig. 5. Real-world experimental setup and evaluation tasks. Tasks: cup-on-table, duck-in-cooker, and towel-folding.

Together, these observations support our central claim: shifting confidence estimation from the action-output space to the frozen representation space yields sharper task-success probabilities at low inference cost, applies across discrete- and continuous-action VLAs, and can become more reliable as online execution evidence accumulates.

C. Robustness under Shifted LIBERO Suites

The standard LIBERO results establish the behavior of VLAConf under the original benchmark distribution. We next evaluate on LIBERO-Pro and LIBERO-Plus to test whether the confidence signal remains useful under systematic distribution shifts, including task, object, layout, camera, robot-state, lighting, background, and sensor perturbations. Following the evaluation protocol above, each shifted suite uses lightweight per-suite Platt calibration before held-out evaluation. To keep the main text focused, Table II reports average online execution results across perturbation families; the full perturbation-level pre-execution and online results are provided in Appendix Tables V and VI.

Table II shows that the main advantage of VLAConf persists under distribution shift after lightweight target-suite calibration. On OpenVLA-OFT, VLAConf and ConfidenceVLA

TABLE III. REAL-WORLD RESULTS.

Task	Success	ECE ↓	Brier ↓	NLL ↓
Cup on Black Table	48/50	0.1011	0.0527	0.2361
Duck into Cooker	43/53	0.0528	0.1545	0.4888
Fold Towel	24/48	0.1584	0.2713	0.7406
Average	115/151	0.1041	0.1595	0.4885

have nearly identical success rates on both shifted suites, yet VLAConf substantially improves the proper scoring rules on both LIBERO-Pro and LIBERO-Plus. The ECE comparison is mixed—slightly worse on LIBERO-Pro but better on LIBERO-Plus—consistent with the standard-LIBERO results, where lower ECE alone does not necessarily indicate sharper per-rollout probabilities. The same representation-level head also remains applicable to the continuous-action $\pi^{0.5}$ backbone, where action-token-based confidence methods are unavailable.

The full results in Appendix Tables V and VI further show that the benefit of online execution evidence largely survives distribution shift. For OpenVLA-OFT, moving from pre-execution to online execution reduces both Brier score and NLL on LIBERO-Pro and LIBERO-Plus. The same pattern appears in Brier score for $\pi^{0.5}$, although its LIBERO-Pro NLL slightly increases. The perturbation-level breakdown also exposes residual failure modes rather than hiding them in the average: for $\pi^{0.5}$, semantic and lighting shifts produce high online NLL despite high success rates, suggesting that a small number of overconfident tail failures can still dominate likelihood under certain shifts. Overall, these shifted-suite results indicate that VLAConf remains informative beyond the original benchmark distribution while preserving the main benefits observed in the standard setting.

D. Real-World Robot Experiments

Finally, we evaluate whether the representation-level confidence signal remains useful beyond simulation by deploying VLAConf on a physical robot platform. As shown in Figure 5, the setup uses a Franka robot arm with a RealSense L515 camera, and demonstrations are collected through a Gello teleoperator. We consider three language-conditioned manipulation tasks: cup-on-table, duck-in-cooker, and towel-folding. These tasks cover rigid, articulated, and deformable manipulation, providing a deployment-facing complement to the benchmark studies above.

Table III summarizes task-success confidence over 151 physical rollouts. VLAConf remains effective in the real-world setting, with average ECE, Brier score, and NLL of 0.1041, 0.1595, and 0.4885 across the three tasks. The cup-on-table task reaches 48/50 successes and obtains the

strongest proper-scoring-rule results, with Brier score 0.0527 and NLL 0.2361. Duck-in-cooker remains well calibrated in terms of ECE (0.0528), but exhibits higher Brier score and NLL. Towel-folding is the most challenging task in this set, achieving 24/48 successes and the largest error on all three calibration metrics. These results show that the confidence signal remains informative on physical rollouts while also reflecting the increased uncertainty of harder real-world manipulation tasks. The corresponding real-world calibration trends across task completion are provided in Appendix Figure 7.

VI. CONCLUSION

We propose VLAConf, a representation-level confidence framework for Vision-Language-Action models that derives calibrated task-success confidence from frozen internal features. Compared with existing confidence methods based on action-token probabilities or repeated sampling, VLAConf estimates confidence in a single forward pass and applies naturally to both discrete- and continuous-action backbones. Experiments on standard LIBERO suites and shifted LIBERO-Pro and LIBERO-Plus benchmarks demonstrate that VLAConf provides more informative task-success probabilities while remaining efficient, and that online evidence can further improve confidence quality during execution. Real-robot experiments further validate VLAConf.

REFERENCES

- [1] K. B. et al., “pi.0: A vision-language-action flow model for general robot control,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.24164>
- [2] —, “pi.0.5: A vision-language-action model with open-world generalization,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.16054>
- [3] M. J. K. et al., “Openvla: An open-source vision-language-action model,” 2025. [Online]. Available: <https://arxiv.org/abs/2406.09246>
- [4] B. Z. et al. (CoRL); Anthony Brohan et al. (arXiv preprint), “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.15818>
- [5] O. M. Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, et al., “Octo: An open-source generalist robot policy,” *arXiv preprint arXiv:2405.12213*, 2024.
- [6] N. B. et al., “Linguistic calibration of long-form generations,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.00474>
- [7] O. E. Stephanie Lin, Jacob Hilton, “Teaching models to express their uncertainty in words,” 2022. [Online]. Available: <https://arxiv.org/abs/2205.14334>
- [8] K. T. et al., “Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback,” 2023. [Online]. Available: <https://arxiv.org/abs/2305.14975>
- [9] C. C. et al., “Inside: LLMs’ internal states retain the power of hallucination detection,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.03744>
- [10] S. F. Lorenz Kuhn, Yarin Gal, “Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation,” 2023.
- [11] C. B. Balaji Lakshminarayanan, Alexander Pritzel, “Simple and scalable predictive uncertainty estimation using deep ensembles,” 2017. [Online]. Available: <https://arxiv.org/abs/1612.01474>
- [12] U. B. Karli, Z. Shanguan, and T. Fitzgerald, “INSIGHT: Inference-time sequence introspection for generating help triggers in vision-language-action models,” 2025.
- [13] J. Kwok, X. Zhang, M. Xu, Y. Liu, A. Mirhoseini, C. Finn, and M. Pavone, “Scaling verification can be more effective than scaling policy learning for vision-language-action alignment,” 2026.
- [14] T. P. Zollo and R. Zemel, “Confidence calibration in vision-language-action models,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.17383>
- [15] R. S. et al., “A system-level view on out-of-distribution data in robotics,” 2022. [Online]. Available: <https://arxiv.org/abs/2212.14020>
- [16] C. X. et al., “Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.08558>
- [17] A. F. et al., “Failure prediction with statistical guarantees for vision-based robot control,” 2022. [Online]. Available: <https://arxiv.org/abs/2202.05894>
- [18] C. A. et al., “Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.04640>
- [19] R. Römer, A. Kobras, L. Worbis, and A. P. Schoellig, “Failure prediction at runtime for generative robot policies,” 2025.
- [20] S. Zhou, B. Zhu, J. Yang, X. Zhao, J. Chen, and Y.-G. Jiang, “RC-NF: Robot-conditioned normalizing flow for real-time anomaly detection in robotic manipulation,” 2026.
- [21] R. Römer, J. Balletshofer, J. Thumm, M. Pavone, A. P. Schoellig, and M. Althoff, “From demonstrations to safe deployment: Path-consistent safety filtering for diffusion policies,” 2025.
- [22] R. S. et al., “Real-time anomaly detection and reactive planning with large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.08735>
- [23] M. K. Cem Gokmen, Daniel Ho, “Asking for help: Failure prediction in behavioral cloning through value approximation,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.04334>
- [24] T. A. et al., “Fighting failures with fire: Failure identification to reduce expert burden in intervention-based learning,” 2020. [Online]. Available: <https://arxiv.org/abs/2007.00245>
- [25] J. D. et al., “Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.00371>
- [26] Q. Gu, Y. Ju, S. Sun, I. Gilitschenski, H. Nishimura, M. Itkina, and F. Shkurti, “Safe: Multitask failure detection for vision-language-action models,” 2025.
- [27] S. Lobel, A. Bagaria, and G. Konidaris, “Flipping coins to estimate pseudocounts for exploration in reinforcement learning,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. PMLR, 2023, pp. 22594–22613.
- [28] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, “FiLM: Visual reasoning with a general conditioning layer,” in *Proceedings of the 32nd AAAI Conference on Artificial Intelligence*, 2018.
- [29] J. Platt, “Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods,” 1999.
- [30] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “Libero: Benchmarking knowledge transfer for lifelong robot learning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 44776–44791, 2023.
- [31] X. Zhou, Y. Xu, G. Tie, Y. Chen, G. Zhang, D. Chu, P. Zhou, and L. Sun, “Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization,” *arXiv preprint arXiv:2510.03827*, 2025.
- [32] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, et al., “Libero-plus: In-depth robustness analysis of vision-language-action models,” *arXiv preprint arXiv:2510.13626*, 2025.
- [33] H. Liu, Y. Zhang, V. Betala, E. Zhang, J. Liu, C. Ding, and Y. Zhu, “Multi-task interactive robot fleet learning with visual world models,” in *Proceedings of the Conference on Robot Learning*, 2024.
- [34] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, “Exploration by random network distillation,” 2018.
- [35] L. Yang, Z. Zhang, Z. Zhang, X. Liu, M. Xu, W. Zhang, C. Meng, S. Ermon, and B. Cui, “Consistency flow matching: Defining straight flows with velocity consistency,” 2024.
- [36] M. J. Kim, Y. Gao, T.-Y. Lin, Y.-C. Lin, Y. Ge, G. Lam, P. Liang, S. Song, M.-Y. Liu, C. Finn, et al., “Cosmos policy: Fine-tuning video models for visuomotor control and planning,” *arXiv preprint arXiv:2601.16163*, 2026.
- [37] L. Li, Q. Zhang, Y. Luo, S. Yang, R. Wang, F. Han, M. Yu, Z. Gao, N. Xue, X. Zhu, et al., “Causal world modeling for robot control,” *arXiv preprint arXiv:2601.21998*, 2026.

APPENDIX I ADDITIONAL ANALYSES

This appendix provides supplementary analyses supporting the main findings while keeping the main text focused. We first examine whether the raw anomaly score reflects execution difficulty before calibration, and then report temporal calibration trends for the real-robot study.

A. Prefix-Score Progress Analysis

Figure 6 analyzes the uncalibrated CFN signal on successful rollouts before Platt scaling. Successful trajectories with larger prefix scores tend to reach their first successful completion later, for both mean and max aggregation. This pattern suggests that the anomaly score captures more than a binary in-/out-of-distribution distinction: even among eventual successes, it reflects apparent execution difficulty or delay.

B. Real-World Calibration Across Task Completion

Figure 7 complements the scalar real-world results in Table III by showing how calibration evolves during execution.

The temporal profiles differ across tasks, with the easiest task remaining comparatively stable and the harder manipulation tasks exhibiting larger residual errors, consistent with the task-wise summary reported in the main paper.

APPENDIX II FULL STANDARD LIBERO RESULTS

Table IV provides the suite-level breakdown underlying the averaged standard-LIBERO comparison in Table I.

APPENDIX III FULL PERTURBATION-LEVEL ROBUSTNESS RESULTS

Tables V and VI provide the perturbation-level breakdown underlying the averaged robustness results in Table II. They report both pre-execution and online-execution calibration for each perturbation family, showing where online-evidence gains are strongest and where residual failure modes remain visible after averaging.

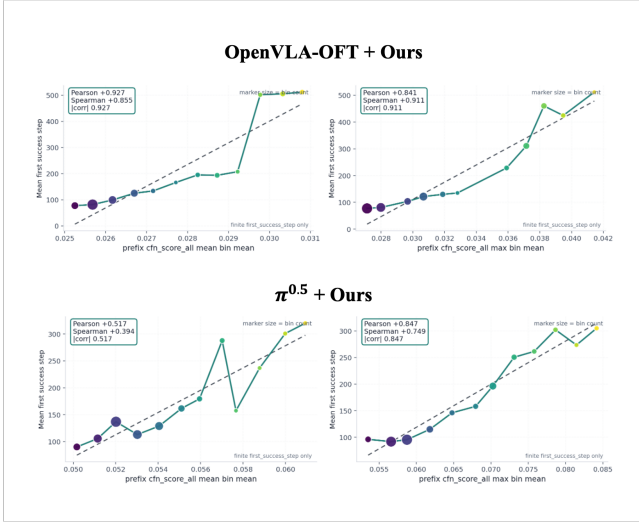


Fig. 6. Prefix-score progress analysis. Successful rollouts are bucketed by their prefix CFN score, using mean and max aggregation, and each marker reports the mean first-success step in that bucket; marker size denotes the bucket count. Higher CFN scores correlate with later first-success steps, suggesting that the uncalibrated anomaly signal tracks execution difficulty.

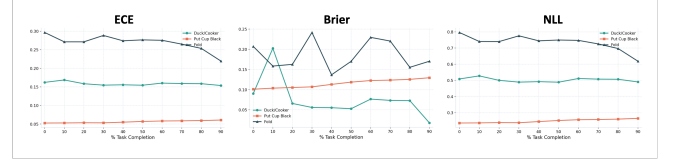


Fig. 7. Real-world calibration trends across task completion for the three evaluation tasks. ECE, Brier score, and NLL are reported at increasing prefix lengths for cup-on-table, duck-in-cooker, and towel-folding.

TABLE IV. FULL STANDARD LIBERO COMPARISON UNDER PRE-EXECUTION AND ONLINE EXECUTION.

Backbone	Suite	Method	Pre-Execution			Online Execution			Succ. (%)	Avg. Inf. Time (ms)
			ECE ↓	Brier ↓	NLL ↓	ECE ↓	Brier ↓	NLL ↓		
OpenVLA-OFT	LIBERO-Spatial	PCA-kmeans	0.2819	0.3373	1.2805	0.4779	0.4650	1.3961	73.0	62.5
		TokenProb	0.2112	0.2315	0.6541	0.0279	0.1758	0.5327	76.0	65.4
		ConfidenceVLA	0.0182	0.1820	0.5498	0.0215	0.1804	0.5448	76.0	710.5
		VLAConf-NoStep	0.0406	0.1925	0.5743	0.0730	0.1907	0.5685	74.0	65.1
		VLAConf	0.0330	0.1880	0.5649	0.2128	0.1196	0.3789	74.6	64.9
	LIBERO-Object	PCA-kmeans	0.2769	0.3083	1.0499	0.2846	0.3122	1.0341	69.2	63.9
		TokenProb	0.1344	0.2402	0.6735	0.0962	0.2239	0.6529	68.8	64.4
		ConfidenceVLA	0.0232	0.2136	0.6183	0.0428	0.2113	0.6130	68.8	711.7
		VLAConf-NoStep	0.0615	0.2181	0.6273	0.0878	0.2096	0.6084	67.6	63.3
		VLAConf	0.0351	0.2071	0.6035	0.1102	0.1950	0.5730	69.8	64.9
	LIBERO-Goal	PCA-kmeans	0.3506	0.2853	0.9624	0.3442	0.2759	0.9205	88.4	64.8
		TokenProb	0.2912	0.1944	0.5779	0.1525	0.1296	0.4356	88.2	65.4
		ConfidenceVLA	0.0674	0.1150	0.4203	0.0186	0.1023	0.3545	88.2	716.5
		VLAConf-NoStep	0.0208	0.0967	0.3428	0.0375	0.0966	0.3420	89.0	65.1
		VLAConf	0.0339	0.0892	0.3289	0.0335	0.0073	0.0485	90.2	64.9
$\pi^{0.5}$	LIBERO-Spatial	PCA-kmeans	0.3623	0.2498	0.7316	0.3835	0.2814	0.8247	90.2	163.0
		ConfidenceVLA	/	/	/	/	/	/	/	/
		VLAConf-NoStep	0.0634	0.0865	0.3399	0.0443	0.0838	0.3107	90.8	164.6
		VLAConf	0.0352	0.0723	0.2768	0.0265	0.0713	0.2703	91.8	164.7
	LIBERO-Object	PCA-kmeans	0.4703	0.4378	1.4658	0.4388	0.4172	1.3327	92.6	164.3
		ConfidenceVLA	/	/	/	/	/	/	/	/
		VLAConf-NoStep	0.0619	0.0909	0.3456	0.0552	0.0888	0.3327	90.8	163.7
		VLAConf	0.0735	0.0835	0.3375	0.0259	0.0759	0.2902	92.2	165.2
	LIBERO-Goal	PCA-kmeans	0.4178	0.3407	1.1173	0.3936	0.3197	1.0315	91.2	163.3
		ConfidenceVLA	/	/	/	/	/	/	/	/
		VLAConf-NoStep	0.0484	0.0919	0.3307	0.0348	0.0926	0.3348	89.8	165.0
		VLAConf	0.0024	0.0906	0.3281	0.1020	0.0531	0.1898	90.0	164.1

TABLE V. FULL LIBERO-PRO ROBUSTNESS RESULTS UNDER PRE-EXECUTION AND ONLINE EXECUTION.

Perturbation	Backbone	Method	Pre-Execution			Online Execution			Success Rate
			ECE ↓	Brier ↓	NLL ↓	ECE ↓	Brier ↓	NLL ↓	
Object	OpenVLA-OFT	ConfidenceVLA	0.0367	0.2265	0.6457	0.0034	0.2245	0.6413	0.6593
	OpenVLA-OFT	VLAConf	0.0354	0.2258	0.6436	0.0157	0.0503	0.1740	0.6467
	$\pi^{0.5}$	VLAConf	0.0400	0.1211	0.4112	0.0303	0.0380	0.1203	0.8626
Position	OpenVLA-OFT	ConfidenceVLA	0.0058	0.2456	0.6844	0.0603	0.2327	0.6586	0.5680
	OpenVLA-OFT	VLAConf	0.0401	0.2405	0.6739	0.0320	0.0513	0.1490	0.5647
	$\pi^{0.5}$	VLAConf	0.0370	0.1454	0.4670	0.0118	0.0242	0.0758	0.8247
Semantic	OpenVLA-OFT	ConfidenceVLA	0.0271	0.1761	0.5370	0.0362	0.1790	0.5466	0.7727
	OpenVLA-OFT	VLAConf	0.0362	0.1741	0.5326	0.0705	0.0756	0.2560	0.7793
	$\pi^{0.5}$	VLAConf	0.0031	0.0332	0.1522	0.0341	0.0341	0.9415	0.9660
Task	OpenVLA-OFT	ConfidenceVLA	0.0176	0.0234	0.1238	0.0021	0.0217	0.1057	0.0220
	OpenVLA-OFT	VLAConf	0.0031	0.0232	0.1125	0.0031	0.0037	0.0258	0.0240
	$\pi^{0.5}$	VLAConf	0.0022	0.0342	0.1515	0.0037	0.0037	0.1023	0.0353
Average	OpenVLA-OFT	ConfidenceVLA	0.0218	0.1679	0.4977	0.0255	0.1645	0.4881	0.5055
	OpenVLA-OFT	VLAConf	0.0287	0.1659	0.4907	0.0303	0.0452	0.1512	0.5037
	$\pi^{0.5}$	VLAConf	0.0206	0.0835	0.2955	0.0200	0.0250	0.3100	0.6722

TABLE VI. FULL LIBERO-PLUS ROBUSTNESS RESULTS UNDER PRE-EXECUTION AND ONLINE EXECUTION.

Perturbation	Backbone	Method	Pre-Execution			Online Execution			Success Rate
			ECE ↓	Brier ↓	NLL ↓	ECE ↓	Brier ↓	NLL ↓	
Layout	OpenVLA-OFT	ConfidenceVLA	0.0010	0.2498	0.6927	0.0381	0.2441	0.6813	0.4864
	OpenVLA-OFT	VLAConf	0.0225	0.2415	0.6758	0.0161	0.0344	0.1401	0.4971
	$\pi^{0.5}$	VLAConf	0.0104	0.1629	0.5083	0.0270	0.0290	0.1087	0.7980
Camera	OpenVLA-OFT	ConfidenceVLA	0.0132	0.0346	0.1649	0.0010	0.0327	0.1468	0.0339
	OpenVLA-OFT	VLAConf	0.0016	0.0311	0.1431	0.0075	0.0075	0.1699	0.0322
	$\pi^{0.5}$	VLAConf	0.0023	0.1370	0.4460	0.0521	0.0383	0.1658	0.1636
Robot Init	OpenVLA-OFT	ConfidenceVLA	0.0019	0.1049	0.3651	0.0056	0.1061	0.3706	0.1193
	OpenVLA-OFT	VLAConf	0.0244	0.1085	0.3806	0.0197	0.0358	0.1674	0.1219
	$\pi^{0.5}$	VLAConf	0.0023	0.1292	0.4265	0.0656	0.0442	0.1925	0.1530
Language	OpenVLA-OFT	ConfidenceVLA	0.0021	0.2049	0.6000	0.0249	0.2059	0.6020	0.7114
	OpenVLA-OFT	VLAConf	0.0023	0.2035	0.5971	0.0268	0.0310	0.1002	0.7149
	$\pi^{0.5}$	VLAConf	0.0877	0.2434	0.6821	0.0291	0.0384	0.1388	0.6334
Light	OpenVLA-OFT	ConfidenceVLA	0.0058	0.2268	0.6458	0.0543	0.2353	0.6648	0.3537
	OpenVLA-OFT	VLAConf	0.0766	0.2320	0.6577	0.0286	0.0352	0.1532	0.3456
	$\pi^{0.5}$	VLAConf	0.0014	0.0356	0.1564	0.0371	0.0371	1.0260	0.9631
Background	OpenVLA-OFT	ConfidenceVLA	0.0201	0.2490	0.6913	0.0990	0.2593	0.7121	0.5006
	OpenVLA-OFT	VLAConf	0.0147	0.2505	0.6941	0.1059	0.0607	0.2165	0.5006
	$\pi^{0.5}$	VLAConf	0.0224	0.1419	0.4579	0.0526	0.0550	0.1612	0.8285
Noise	OpenVLA-OFT	ConfidenceVLA	0.0296	0.1434	0.4626	0.0254	0.1428	0.4606	0.1719
	OpenVLA-OFT	VLAConf	0.0096	0.1417	0.4573	0.0146	0.0230	0.1197	0.1710
	$\pi^{0.5}$	VLAConf	0.0095	0.1636	0.5087	0.0399	0.0462	0.1840	0.2066
Average	OpenVLA-OFT	ConfidenceVLA	0.0105	0.1733	0.5175	0.0355	0.1752	0.5197	0.3396
	OpenVLA-OFT	VLAConf	0.0217	0.1727	0.5151	0.0313	0.0325	0.1524	0.3405
	$\pi^{0.5}$	VLAConf	0.0194	0.1448	0.4551	0.0433	0.0412	0.2824	0.5352